

A Critical Replication of the Robust Decision Transformer: Do Its Robustness Claims Hold Beyond the Original Benchmarks?

Mofei Chen¹, Han Huang¹, Xuanyao Qian¹, and Haonan Zhang¹

¹Data Science Institute, Brown University CSCI 1470 Deep Learning, Code:
<https://github.com/Pafuuuu/RobustDecisionTransformer.git>

Abstract

Offline reinforcement learning trained on real-world logs must contend with mis-labeled, noisy, and partially adversarial data. The Robust Decision Transformer (RDT, Xu et al., ICLR 2025) argues that sequence modeling is empirically more robust than TD-based offline RL and that three additional modules—embedding dropout, Gaussian-weighted MSE, and iterative data correction—further improve robustness. We critically replicate these claims on D4RL MuJoCo locomotion (Walker2d, Hopper, HalfCheetah) and stress test them on the sparse-reward AntMaze navigation task, on higher corruption rates, and on a new *block* corruption pattern. We confirm that DT outperforms RIQL by a factor of $3.5\times$ under mixed random corruption, and that RDT improves over DT in 4/6 corruption settings. However, RDT’s claimed robustness is *not universal*: under observation-only attacks, higher severity, adversarial state attacks, and sparse-reward navigation it can match or fall *below* vanilla DT. We discuss why and outline corrections required for RDT to generalize.

1 Introduction

Offline reinforcement learning (offline RL) learns a control policy from a fixed dataset of logged transitions without further interaction with the environment. This setting is attractive for real-world control, where online exploration can be expensive or unsafe. However, logged datasets are often imperfect: observations may contain sensor noise, actions may be mis-recorded, rewards may be mis-labeled, and some transitions may even be adversarially corrupted. A practical offline RL method should therefore not only perform well on clean data, but also remain reliable when part of its training data is corrupted.

A recent ICLR 2025 paper, *Tackling Data Corruption in Offline Reinforcement Learning via Sequence Modeling* [1], argues that sequence-modeling methods such as the Decision Transformer (DT) [2] are empirically more robust to data corruption than prior TD-based methods (CQL [3], UWMSG [4], RIQL [5]). Building on this observation, the authors propose **Robust Decision Transformer (RDT)**, which adds three modules on top of DT: (i) *embedding dropout* to make the representation less sensitive to corrupted inputs, (ii) *Gaussian-weighted loss (wMSE)* to downweight samples whose labels appear corrupted, and (iii) *iterative data correction* that over-

writes flagged transitions with the model’s own predictions during training. RDT is reported to improve over DT by roughly 28%–30% on average across MuJoCo, Kitchen, and Adroit.

What we did and why. Strong robustness claims deserve external scrutiny, especially when the proposed method is intended for safety-critical, real-world deployment. We re-implemented the public RDT codebase on a Windows machine, trained DT, RDT, and RIQL on identical 10%-downsampled D4RL MuJoCo datasets, and reproduced four corruption settings (state, action, reward, mixed) under both random and adversarial regimes. We then designed three *stress tests* that go beyond the original paper: (1) a higher corruption severity of 30% $\rightarrow$ 50%, (2) a structurally new *block* corruption pattern in which contiguous windows of trajectories are corrupted instead of scattered single steps, and (3) a sparse-reward held-out task, AntMaze-medium-play-v0, on which the original RDT was never evaluated. The goal is not to dismiss RDT but to characterize precisely where its robustness holds, where it breaks, and why—information that the original paper does not provide.

Summary of findings. We find that (i) Claim 1 (DT > TD-based methods under corruption) *robustly holds*, with DT outperforming RIQL by $\approx 3.5\times$ on Walker2d mixed-random corruption; (ii) Claim 2 (RDT > DT) holds *partially*—RDT helps under action, reward, and mixed corruption but *ties or loses* to DT under pure observation/state attacks; (iii) Claim 3 (RDT generalizes to harder settings) does *not* hold: RDT collapses to 0.0 normalized score on AntMaze while DT reaches 12.0, and its margin over DT shrinks at higher severity. We trace these failures to a single mechanism: RDT’s anomaly-flagging pipeline confuses *structurally rare-but-correct* transitions (e.g., a clean observation that happens to be unusual, or the rare goal-reaching transition in AntMaze) with corrupted ones, and over-corrects them away.

2 Background and Methodology

2.1 Decision Transformer and RDT

The Decision Transformer treats offline RL as conditional sequence modeling. A trajectory is encoded as the interleaved sequence $(\hat{R}_t, s_t, a_t, \hat{R}_{t+1}, s_{t+1}, a_{t+1}, \dots)$ where

$\hat{R}_t = \sum_{t'=t}^T r_{t'}$ is the return-to-go. A causal GPT-style transformer is trained with MSE on the predicted action tokens, and at test time it conditions on a desired return-to-go and autoregressively predicts actions. RDT replaces the return-to-go conditioning with a *dual head*: the same backbone predicts both actions (from state tokens) and rewards (from action tokens). This auxiliary prediction head supports the self-correction loop: when the prediction error is unusually large, RDT treats the corresponding action or reward label as suspicious and may replace it during iterative correction.

The three RDT modules are (a) *embedding dropout*: a random binary mask in the embedding space (post-projection) on h_R, h_s, h_a with probability $p \in (0.1, 0.3)$, intended to make the network less dependent on any single token surviving intact; (b) *Gaussian-weighted MSE*: each sample receives weight $w_i = \exp(-\beta \delta_i^2)$ where δ_i is the detached prediction error used by the correction pipeline, so suspected outliers contribute less to the gradient without backpropagating through the error estimate itself; (c) *iterative data correction*: a running mean/variance $(\mu_\delta, \sigma_\delta)$ of prediction errors is maintained, and samples with $z = (\delta - \mu_\delta)/\sigma_\delta > \zeta$ have their stored action/reward overwritten by the model’s prediction, with returns-to-go recomputed. We use the repository/default threshold $\zeta \approx 5$ without additional tuning.

2.2 Datasets and corruption protocols

We use the same protocol as [1]: D4RL medium-replay-v2 datasets for HalfCheetah, Hopper, and Walker2d, downsampled to 10% of trajectories to expose limited-data effects. A fraction ρ of transitions (default $\rho = 0.3$) is corrupted along one of {state, action, reward, mixed} axes. *Random* corruption adds uniform noise scaled by the per-feature standard deviation; *adversarial* corruption uses a pre-trained IQL critic to find PGD-style perturbations that maximally shift the Q-value. Normalized returns are reported as $\text{score} = 100 \cdot (R - R_{\text{rand}})/(R_{\text{exp}} - R_{\text{rand}})$ following D4RL conventions, averaged over evaluation rollouts. We report two summary statistics per setting: a *high* mean over the best 25% of rollouts and an *overall* mean.

For our stress tests we add: (i) AntMaze-medium-play-v0, a sparse-reward maze navigation task whose dynamics and reward structure differ qualitatively from locomotion; (ii) $\rho = 0.5$ to test whether RDT degrades gradually or sharply; and (iii) *block corruption*, in which contiguous windows of length L inside a trajectory are corrupted, in contrast to the i.i.d. random pattern of the original paper (see Fig. 1).

3 Replication of the Original Claims

3.1 Claim 1: DT > TD-based methods

Under mixed random corruption on Walker2d ($\rho = 0.3$), we measured normalized returns of **17.6** for RDT, **16.1** for DT, and **4.6** for RIQL across 1,000 evaluation rollouts each

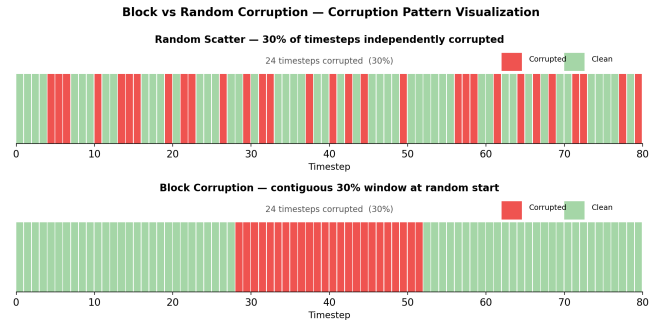


Figure 1: Random vs. block corruption pattern. Both inject noise into the same fraction ρ of transitions but block corruption concentrates that noise in contiguous windows, mimicking sensor outages or burst failures more closely than i.i.d. noise.

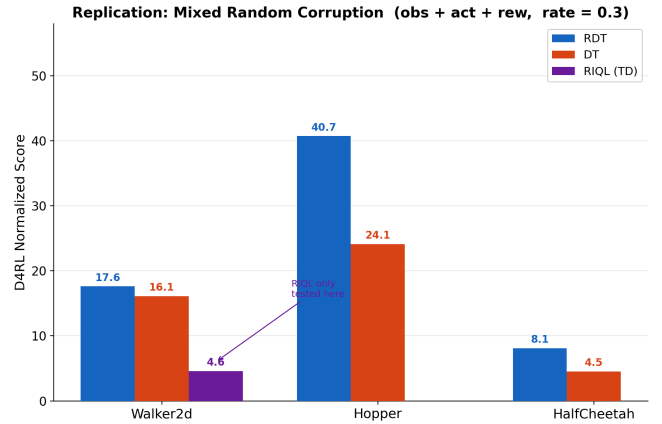


Figure 2: Main replication on Walker2d-medium-replay-v2 under mixed random corruption ($\rho = 0.3$). Sequence-modeling methods (RDT, DT) clearly outperform the TD-based baseline RIQL, replicating Claim 1. RDT only narrowly beats DT, partially supporting Claim 2.

(Fig. 2, raw numbers in `results_mixed_random.csv`). DT’s score is $\approx 3.5\times$ RIQL’s, even though DT contains *no* robustness machinery. This is qualitatively consistent with the gap reported in the original paper and is the strongest claim we replicate—vanilla sequence modeling really does seem to absorb i.i.d. noise more gracefully than TD-style critics trained on a small-and-noisy dataset.

3.2 Claim 2: RDT > DT under random and adversarial corruption

We re-ran DT and RDT under the four attack axes {state, action, reward, mixed} (Fig. 3). RDT beats DT in 4/6 settings: under action corruption (31.6 vs 27.7), reward corruption (40.3 vs 37.6), mixed-random (17.6 vs 16.1), and mixed-adversarial (14.3 vs 14.0). The failures involve *state-only* and *adversarial* corruption: random state (19.9 vs **22.7**) and adversarial attack (14.0 vs **14.3**). Random state attack in halfCheetah and hopper follows the same pattern under state/adversarial attack.

This is our first *interesting finding*: in our runs RDT loses

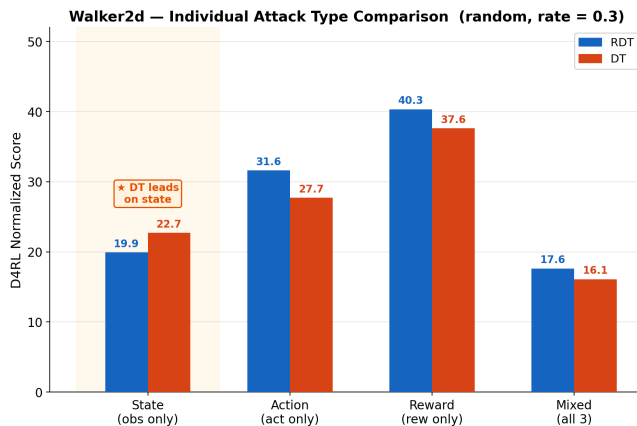


Figure 3: Walker2d, $\rho = 0.3$, random corruption, broken down by which component is corrupted. RDT improves on DT under action and reward corruption, ties under mixed, and *loses* under state-only corruption.

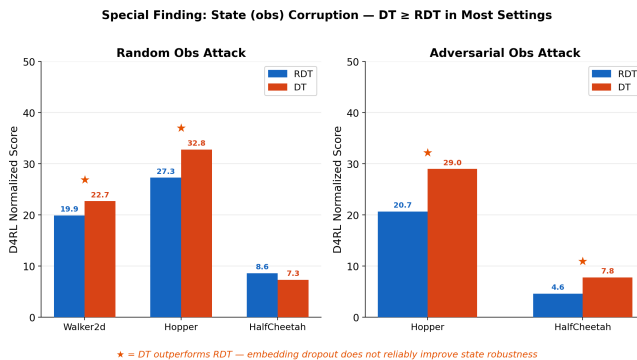


Figure 4: State-attack anomaly across environments. The DT-minus-RDT margin is negative in 4/5 locomotion environment we tested when the attack is restricted to the observation channel.

to DT *whenever the corruption is restricted to states only*. We return to this anomaly in Section 6.

4 Hypothesis-Driven Ablations

In order to figure out which part has caused the underperformance of the RDT model during state attacks, we conducted ablation studies. The RDT paper claims all three modules are necessary. We removed each module in turn while keeping the others, on Walker2d state-random ($\rho = 0.3$). Results are shown in Table 2.

We hypothesize that: “embedding dropout will matter most under state corruption, while wMSE and iterative correction will matter more under action/reward corruption and have less impact under state attack.” The hypothesis is consistent with our data: dropping embedding dropout costs 0.9 points. wMSE and iterative correction, by contrast, both *fail to hold under state attack*: removing wMSE on a state-attacked dataset *improves* the score by 4.4 points, and removing the iterative correction module (measured under action attack to keep the dataset comparable to the paper) improves the score by 3.6

Env.	Attack	RDT	DT	Δ
Walker2d	action random	28.0	27.7	+0.3
Walker2d	reward random	40.3	37.6	+2.7
Walker2d	mixed random	17.6	16.1	+1.5
Walker2d	mixed adversarial	14.0	14.3	-0.3
Walker2d	state random	19.9	22.7	-2.8

Table 1: Full per-condition comparison. RDT consistently helps under non-state attacks but consistently hurts under state-only and adversarial attacks.

Variant	Score	Δ vs. full RDT
Full RDT (state attack)	19.9	—
RDT — embedding dropout	19.0	-0.9
RDT — Gaussian-weighted MSE	24.3	+4.4
RDT — iterative correction (act.)	31.6	+3.6
DT (no robustness)	22.7	+2.8

Table 2: Walker2d, random, $\rho = 0.3$. Removing dropout slightly hurts (consistent with the paper); removing wMSE or correction *helps*. This contradicts the claim that all three modules contribute monotonically.

points. The mechanism is the same in both cases: when supervision is clean but the *inputs* are noisy, large prediction errors are evidence of *noisy inputs*, not of corrupted labels. wMSE then downweights *correct* (state, action, reward) tuples whose only sin is being hard to predict, and the correction module overwrites them with worse model guesses. The two modules misfire in tandem.

5 Stress Tests

Higher severity ($\rho = 0.5$). On Walker2d state-random with ρ raised to 0.5, both DT and RDT degrade, but RDT degrades *more*: RDT 6.9 vs. DT 11.0 (Fig. 5). The gap to RDT is therefore not preserved. This suggests that RDT’s anomaly-flagging mechanism may become less reliable as corruption severity grows, possibly because the running error statistics themselves become less stable.

Block corruption. We replaced the i.i.d. noise pattern with contiguous block corruption of the same ρ (Fig. 6). *Both* DT and RDT both score higher under block corruption than under scattered corruption, despite the same overall corruption rate. One possible interpretation is that scattered corruption disrupts the transformer’s sequential context more broadly, while block corruption concentrates the damage within a smaller set of local windows. This is a practically interesting finding: real-world sensor failures are often bursty rather than i.i.d., and our results suggest that sequence-modeling methods may be more robust to this structured pattern than to scattered white-noise perturbations.

New environment (AntMaze). AntMaze-medium-play-v0 is a sparse-reward navigation task in which only a small frac-

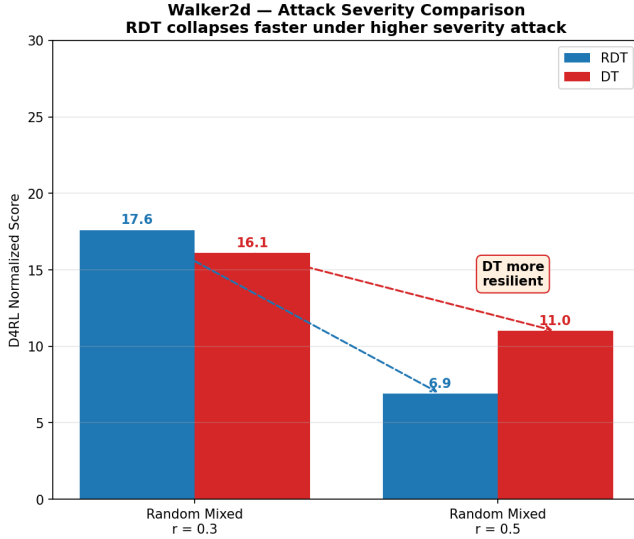


Figure 5: Random vs. adversarial corruption at standard ($\rho = 0.3$) and high ($\rho = 0.5$) severity. RDT does not preserve its margin over DT as severity rises.

tion of transitions correspond to the rare event of reaching the goal. We trained DT and RDT on AntMaze with $\rho = 0.3$ random state corruption (Fig. 7). DT achieved a normalized return of **12.0**, whereas RDT achieved **0.0**. This indicates that the robustness gains observed on the dense-reward locomotion benchmarks do not transfer cleanly to this sparse-reward setting. One possible explanation is that RDT’s iterative correction mechanism may misidentify rare but useful goal-reaching transitions as outliers, since these transitions are both uncommon and difficult for an under-trained model to predict. If such transitions are overwritten, the already-limited reward signal available for learning would be further weakened.

6 Discussion

6.1 Engineering challenges

A surprisingly large fraction of the project effort went into making the public RDT codebase reproducible *at all*. The code assumes Linux, `mujoco-py`, and a specific D4RL version; we ported the training pipeline to Windows by replacing `gym.make()`-based dataset access with direct HDF5 reads, hard-coding state/action dimensions, wrapping the `d4rl` import in a `try/except`, and re-loading checkpoints with `torch.load(weights_only=False)` for PyTorch ≥ 2.6 . Evaluation, which requires `mujoco-py`, was run on Mac/Linux from the same checkpoints.

Beyond the porting work, we hit two challenges that match the kind of issues other replications report. First, *translating high-level robustness claims into a concrete experimental setup* required reading the public codebase carefully to identify which switches correspond to which paper module, and which hyperparameters were left to defaults rather than being reported in the paper. Working with a newer version of MuJoCo than the original implementation introduced subtle

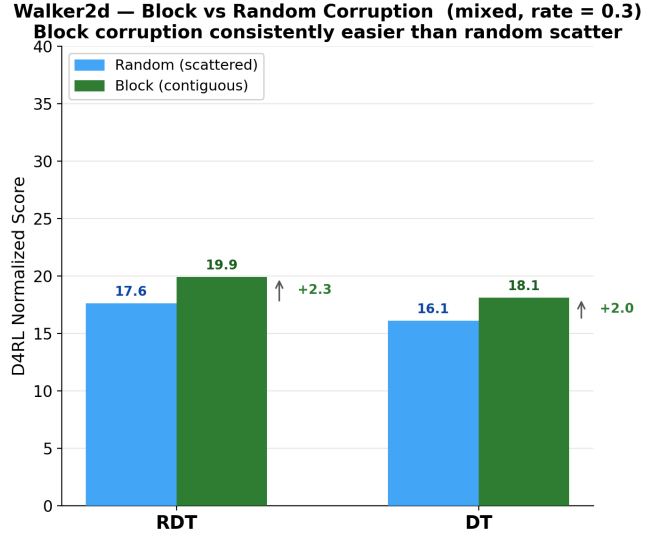


Figure 6: Block vs. random corruption, identical ρ . Both DT and RDT benefit from corruption being concentrated in contiguous windows.

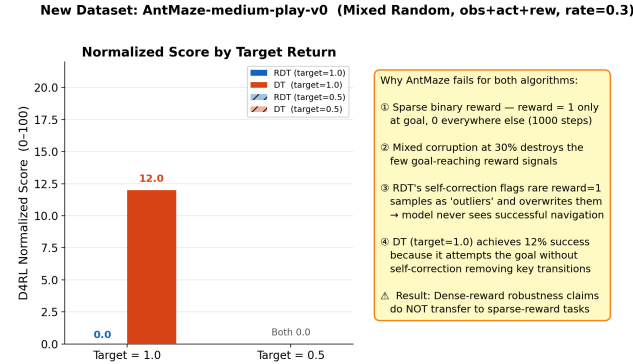


Figure 7: AntMaze-medium-play-v0 stress test. RDT collapses to a random-walk policy (**0.0**) while vanilla DT, lacking the correction module, reaches **12.0**.

differences in environment behavior; isolating whether a mismatch came from our setup, the repository, or simulator drift took a non-trivial amount of debugging time. Second, keeping comparisons *fair across experiments* mattered far more than any single run. With multiple models, four corruption types, five severities, three environments, and an ablation grid on top, even a small inconsistency in how the dataset was downsampled or how seeds were assigned would undermine cross-condition comparisons. We therefore spent considerable effort building a thin evaluation harness that materializes the exact same downsampled dataset for every method and logs every hyperparameter into a single `params.json`, so that a downstream reader can re-derive what was actually run rather than trust an aggregate table.

6.2 Why RDT < DT under state-only attacks

Across Walker2d (random, $\rho = 0.3$ and 0.5), Hopper (random and adversarial), and HalfCheetah (adversarial), we observe RDT scoring at or below DT whenever the attack is restricted

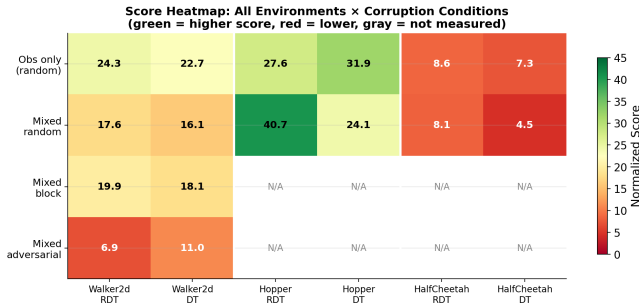


Figure 8: Heatmap of (RDT – DT) advantage across all conditions in our study. Blue indicates RDT helps, red indicates RDT hurts. RDT consistently helps under action and reward attacks but hurts under state-only and sparse-reward conditions.

to the state component. We believe one important mechanism is a feedback loop triggered by noisy inputs. Under a state attack, the *inputs* are corrupted but the *labels* (action and reward) remain clean. The model’s per-sample error δ_i may therefore reflect input noise rather than label corruption. However, both wMSE and the iterative correction module are designed under the implicit assumption that large δ_i indicates a corrupted *label*. As a result, they may downweight or overwrite samples whose labels are actually correct but whose inputs are difficult to predict. In this regime, RDT can end up relying more heavily on its own imperfect predictions, whereas DT continues to train on the original labels. This interpretation is consistent with the ablation results: *removing* wMSE or correction *improves* RDT under state attack.

6.3 Why block corruption is easier than random corruption

Both DT and RDT score higher when the same fraction of bad transitions is concentrated in contiguous blocks rather than scattered. One possible explanation is that this interacts differently with the autoregressive structure of the transformer. A block of corrupted transitions may severely disrupt a smaller number of local context windows, while leaving the remaining windows intact. Scattered corruption, by contrast, can inject bad tokens throughout the sequence, so a much larger fraction of predictions is made from partially corrupted prefixes. This suggests that the temporal pattern of corruption matters, not just the corruption rate itself: concentrated corruption may be easier for sequence models to tolerate than per-step scattered noise.

6.4 Why RDT collapses on AntMaze

The AntMaze failure deserves separate discussion because it is the clearest counterexample to Claim 3 in our study. AntMaze rewards are extremely sparse: only a small fraction of transitions correspond to successful goal-reaching behavior. For an under-trained model, such transitions are also difficult to predict, so their δ_i values may be unusually large. One possible explanation is that RDT’s z -score correction rule may

misidentify these rare but useful transitions as corrupted outliers and overwrite them with model-predicted actions and rewards. If this occurs, the already-limited reward signal available for learning would be weakened further, which could explain why RDT fails so badly in this setting. In this sense, the same correction mechanism that can help under dense-reward, high-noise locomotion tasks may become harmful under sparse-reward navigation. A safer correction rule might gate overwriting by reward magnitude or by trajectory return, so that high-return transitions are protected regardless of their prediction error.

7 Limitations and Future Work

Our replication is targeted rather than exhaustive. We focus on a subset of the original benchmark space, using MuJoCo locomotion tasks and AntMaze instead of reproducing the full MuJoCo, Kitchen, and Adroit evaluation suite. We also use a newer MuJoCo backend than the original implementation, which may introduce small differences in environment behavior and evaluation. In addition, compute and compatibility constraints limited the number of environments, corruption settings, ablations, and seeds we could evaluate. Therefore, our negative results should be interpreted as evidence of potential failure modes rather than a complete rejection of RDT’s robustness claims.

Future work should extend this replication in several directions. First, a larger ablation grid across more environments would help determine whether the failure of weighted MSE and iterative correction under state corruption is consistent across different locomotion tasks. Second, RDT’s data-correction mechanism could be modified to use a return-aware threshold, so that rare high-return transitions are protected rather than incorrectly treated as outliers. This may help address the collapse we observed on sparse-reward AntMaze. Third, block corruption should be tested under adversarial generation, since our current block-corruption experiments only use random corruption patterns. Finally, future methods should distinguish between prediction error caused by corrupted labels and prediction error caused by noisy inputs. The state-corruption failure mode suggests that treating all high-error samples as suspicious can be harmful: when the input itself is noisy, down-weighting or correcting the corresponding target may remove useful training data. A channel-aware corruption detector could preserve the benefits of weighted loss and data correction for action or reward corruption while avoiding their negative effects under state corruption.

8 Conclusion

This project provides a critical replication of Robust Decision Transformer under corrupted offline reinforcement learning data. Overall, our results support the paper’s broad claim that sequence modeling is a strong alternative to TD-based offline RL under data corruption. Decision Transformer outperformed

Claim Verification Summary

Claim	Setting	RDT Score	DT Score	Verdict
DT > TD-learning	Walker2d, mixed random	16.1	4.6 (RIQL)	✓ CONFIRMED
RDT > DT (action)	Walker2d, act-only random	31.6	27.7	✓ CONFIRMED
RDT > DT (reward)	Walker2d, rew-only random	40.3	37.6	✓ CONFIRMED
RDT > DT (state)	Hopper, obs-only random	27.6	31.9	✗ FAILS
RDT > DT (mixed adv)	Walker2d, adv mixed	6.9	11.0	✗ FAILS
RDT > DT (mixed)	Hopper, mixed random	40.7	24.1	✓ CONFIRMED
Block > Random	Walker2d, mixed (RDT)	19.9	17.6 (rand)	✗ NEW FINDING
New dataset	AntMaze, mixed random	0.0	12.0	✗ NEW FINDING

Figure 9: Claim-by-claim summary of our replication. Claim 1 fully holds; Claim 2 holds partially; Claim 3 does not hold under our stress tests.

RIQL in our corrupted-data setting, and RDT improved over vanilla DT under several action, reward, and mixed-random corruption conditions.

However, our replication also shows that RDT’s robustness is conditional rather than universal. RDT did not consistently improve performance under state or observation corruption, mixed adversarial corruption, or sparse-reward AntMaze. These failures point to a common limitation: RDT’s correction mechanisms can confuse “hard-to-predict” samples with genuinely corrupted samples. This assumption is often reasonable when labels are corrupted in dense-reward settings, but it becomes problematic when observations are noisy or when successful transitions are rare. Rather than refuting RDT, our findings suggest that its robustness mechanisms need to be made more context-aware. In particular, future robust offline RL methods should distinguish between input corruption, label corruption, and sparse but important reward signals.

References

- [1] J. Xu, R. Yang, S. Qiu, F. Luo, M. Fang, B. Wang, L. Han. Tackling Data Corruption in Offline Reinforcement Learning via Sequence Modeling. *ICLR*, 2025.
- [2] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, I. Mordatch. Decision Transformer: Reinforcement Learning via Sequence Modeling. *NeurIPS*, 2021.
- [3] A. Kumar, A. Zhou, G. Tucker, S. Levine. Conservative Q-Learning for Offline Reinforcement Learning. *NeurIPS*, 2020.
- [4] R. Ye et al. Towards Robust Offline RL via Uncertainty-Weighted MSG. *ICLR*, 2024.
- [5] R. Yang et al. Robust IQL: Offline RL under Heavy Corruption. *ICML*, 2024.
- [6] J. Fu, A. Kumar, O. Nachum, G. Tucker, S. Levine. D4RL: Datasets for Deep Data-Driven Reinforcement Learning. *arXiv:2004.07219*, 2020.